

TEMA 2 – Pantallas

Objetivo del tema

En este tema trataremos la complejidad e importancia que tiene la pantalla del dispositivo móvil a la hora de diseñar nuestras App.

2.1 El problema de las pantallas

Cuando se ‘inventó’ Android, sólo existía un terminal capaz de soportarlo. Desde entonces, Android se ha instalado en todo tipo de dispositivos, desde móviles a tablets pasando por relojes inteligentes, etc. Esto significa que **no podemos** hacer una App pensando en que se va a ver en un dispositivo en concreto. Si lo hacemos, estamos restringiendo nuestra a que nuestra App no se vea bien en un alto % de dispositivos.

Como norma general, todo lo que programes tiene que poder verse en cualquier pantalla.

Las diferentes resoluciones estándar de Android son las siguientes:

Abreviatura	Nombre completo	Resolución (px)	Aspect ratio aproximado
QVGA	Quarter VGA	240 × 320	3:4
WQVGA	Wide QVGA	240 × 400	3:5
HVGA	Half VGA	320 × 480	2:3
WVGA	Wide VGA	480 × 800	3:5
FWVGA	Full Wide VGA	480 × 854	16:9
SVGA	Super VGA	600 × 800	3:4
DVGA	Double VGA	960 × 640	3:2
QHD	Quarter HD	540 × 960	16:9
HD	High Definition	720 × 1280	16:9
WXGA	Wide XGA	1280 × 800	16:10
FHD / 1080p	Full HD	1080 × 1920	16:9
WQHD / 2K	Wide Quad HD	1440 × 2560	16:9
QHD+ / 2K+	Quad HD Plus	1600 × 2560	16:10
UHD / 4K	Ultra HD	2160 × 3840	16:9
5K / 4K+	Ultra HD+	2880 × 5120	16:9

Obviamente, esta clasificación se va actualizando según la tecnología va avanzando.

Para poder desarrollar una App que se vea bien en cualquier dispositivo (pantalla), se recurren a una serie de ‘trucos’. Que son en realidad características de la propia Arquitectura de Android. El propio Sistema Operativo tiene claramente separados cada uno de sus componentes; y esa misma idea se ha trasladado a las App. Es por eso por lo que el código, el interfaz y los recursos están separados.

No nos tiene que extrañar por tanto que existan formas de trabajar con cosas tan específicas como las diferentes resoluciones de pantalla de forma simple e independiente del código.

2.2 Trabajando con diferentes pantallas

La solución pasa por utilizar **layouts adaptativos** y **recursos específicos** según la densidad y tamaño requeridos.

- Cada vez que hagamos una versión de un recurso específico para un tipo de pantalla, debemos colocarlo en una subcarpeta concreta dentro de su categoría. Estas carpetas se definen encadenando diferentes **sufijos** dando lugar a un nombre identificativo.

Los criterios son:

- **Tamaño de pantalla** (small, normal, large, xlarge)
- **Orientación** (port = vertical, land = horizontal)
- **Densidad de pantalla** (ldpi, mdpi, hdpi, xhdpi, xxhdpi, xxxhdpi)

De esta forma podríamos tener:

res/layout/main_activity.xml: Ruta para el layout por defecto

res/layout-large/main_activity.xml: Ruta para las pantallas grandes

res/layout-land/main_activity.xml: Ruta para landscape

res/layout-sw600dp/main_activity.xml: Ruta para pantallas $\geq 600dp$ de ancho

Esta lógica se extiende también a las imágenes (Drawables):

res/drawable-hdpi/imagen.png: Ruta para imágenes para pantallas hdpi

res/drawable-xhdpi/imagen.png: Ruta para imágenes para pantallas xhdpi

El propio Android Studio dispone herramientas en su IDE para facilitar estas tareas. Por ejemplo, si añadimos una imagen cualquiera a nuestra App, el propio Android Studio nos permite convertirla para todas las resoluciones automáticamente.

Tampoco es necesario que indiquemos el tipo de pantalla de ninguna forma concreta. Cuando la App se instala en el dispositivo y la arrancamos, el propio Sistema Operativo selecciona automáticamente la carpeta correcta según el dispositivo. Obviamente, siempre recurrirá a la carpeta por defecto si no tiene una adecuada.

- Existe un fichero `res/values/dimens.xml` que define valores de tamaño reutilizables para toda la App (márgenes, tamaños de texto, etc.). Podemos usarlo para mantener la consistencia de la UI y facilitar la adaptación a diferentes tamaños de pantalla. Por ejemplo, podemos definir aquí un margen de 20dp y denominarlo 'margen1'. Ahora, cualquier widget de nuestra App puede utilizar 'margen1' como una medida, en lugar de tener que ser indicada de forma explícita.

- Podemos usar **Layouts Adaptativos** como el ConstraintLayout en el layout por defecto, que permiten anclar diferentes widgets entre sí. Teóricamente, un uso correcto de este layout no requeriría tener que diseñar diferentes layouts... no obstante, los resultados

Actualmente Android recomienda utilizar **Jetpack Compose**, una herramienta que simplifica la creación de interfaces gráficas.

2.2 Listado de Sufijos para /res

Cuando hemos introducido el tema de los Sufijos, hemos mencionado un par de ejemplos simples. Pero en realidad los sufijos se pueden encadenar para especificar cosas con mucho detalle. A continuación, se presentan las tres categorías que existen:

Tamaño de pantalla: Tamaño relativo a un terminal “estándar”	
Sufijo	
-small	Tamaño menor a 3 pulgadas
-medium	Un móvil normal, desde las 3 hasta las 4.5 pulgadas
-large	Para móviles grandes o tablets pequeñas, desde 4.5 hasta 7 pulgadas
-xlarge	Para tablets, notebooks, o televisiones. Originalmente de 7 a 10 pulgadas

Densidad: Densidad de píxeles por pantalla. Se mide en puntos por pulgada (dpi)	
Sufijo	
-ldpi	Para pantallas desde los 100 hasta los 150 dpi
-mdpi	Para pantallas desde los 150 hasta los 200 dpi
-hdpi	Para pantallas desde los 200 hasta los 250 dpi
-xhdpi	Para pantallas desde los 250 hasta los 300 dpi
-xxhdpi	Para pantallas de alrededor de los 480 dpi
-xxxhdpi	Para pantallas de alrededor de los 640 dpi
-nodpi	Para recursos que no deberían ser escalados sin importar la pantalla

Relación de Aspecto: Relación de la altura con respecto a la anchura de la pantalla	
Sufijo	
-long	Para pantallas más altas que los dispositivos estándar
-notlong	Para pantallas de relación de aspecto estándar

Cada uno de estos sufijos se puede usar de forma independiente o en combinación, lo que nos permite establecer rutas cosas como:

```
res/layout-small-long/  
res/layout-large/  
res/drawable-hdpi/
```

Normalmente utilizaremos las variantes de densidad para los *drawables*, y el tamaño y relación de aspecto para los *layouts*. Sin embargo, vale cualquiera de ellos.

2.3 Pantallas, densidades y tamaños

Es posible encontrar por internet tablas de referencia como la que ves aquí en la que se puede consultar la relación que hay entre los modelos de pantalla, densidades y tamaños. Esta tabla probablemente esté obsoleta, la incluyo por referencia.

	Low density (120), <i>ldpi</i>	Medium density (160), <i>mdpi</i>	High density (240), <i>hdpi</i>	Extra high density (320), <i>xhdpi</i>
Small screen	QVGA (240x320)		480x640	
Normal screen	WQVGA400 (240x400) WQVGA (240x432)	HVGA (320x480)	WVGA (480x800) WVGA854 (480x854) 600x1024	640x960
Large screen	WVGA800 (480x800) WVGA854 (480x854)	WVGA800 (480x800) WVGA854 (480x854) 600x1024		
Extra large screen	1024x600	WXGA (1280x800) 1024x768 1280x768	1536x1152 1920x1152 1920x1200	2048x1536 2560x1536 2560x1600

2.4 Configuraciones de pantalla soportadas

Puede ser que una App no pueda mostrarse en ciertas pantallas. Este tipo de cosas puede ser indicada mediante el **AndroidManifest.xml** añadiendo restricciones:

```
<supports-screens android:smallScreens="false"  
                  android:normalScreens="true"  
                  android:largeScreens="true"  
                  android:anyDensity="true" />
```

Poner false fuerza al Android a mostrar las interfaces indicadas mediante un modo de compatibilidad que, en realidad, es un escalado. **No siempre funciona**, pero por lo menos se intenta.

Ejercicio 9

Repite el ejercicio 6 que simula una llamada telefónica a Walter White, pero haciendo que las imágenes estén disponibles en todas las densidades. Crea también al menos dos layout diferentes para diferentes resoluciones de pantalla. Finalmente, despliega la App en diferentes dispositivos (móvil y tablet) y observa los cambios.